

Systemy rekomendacji personalizowanych na niewielkich zbiorach danych: analiza na przykładzie książek i muzyki

Maciej Andrzejewski
Politechnika Poznańska
Poznań, Polska
maciej.andrzejewski
@student.put.poznan.pl

Wojciech Kaczmarek
Politechnika Poznańska
Poznań, Polska
wojciech.kaczmarek.1
@student.put.poznan.pl

Julia Nowakowska
Politechnika Poznańska
Poznań, Polska
julia.nowakowska
@student.put.poznan.pl

Sławomir Stefański
Politechnika Poznańska
Poznań, Polska
slawomir.stefanski
@student.put.poznan.pl

Sebastian Wróbel
Politechnika Poznańska
Poznań, Polska
sebastian.wrobel
@student.put.poznan.pl

Opis

W niniejszej pracy przedstawiono proces projektowania i implementacji spersonalizowanego systemu rekomendacji muzycznych działającego na małych zbiorach danych (*small datasets*). Głównym celem projektu było zbudowanie modelu klasyfikacyjnego, który na podstawie prywatnych playlist użytkowników z serwisu Spotify potrafi trafnie wyselekcjonować i polecić nowe utwory z globalnej bazy danych liczącej 114 tysięcy pozycji.

W ramach badań przeprowadzono dwie iteracje eksperymentalne. Pierwsza z nich, oparta wyłącznie na fizycznych parametrach akustycznych dźwięku pozyskanych przez Spotify API, wykazała tendencję modeli do zamykania użytkowników w tzw. bańce filtrującej, oferując utwory monotonne i bardzo zbliżone brzmieniowo. W drugiej iteracji wdrożono autorski potok ETL integrujący dane ze Spotify z zewnętrzną bazą MusicBrainz, co pozwoliło wzbogacić model o kontekst geograficzny (kraj pochodzenia artysty).

Do celów klasyfikacji wykorzystano algorytmy Regresji Logistycznej oraz Lasu Losowego. W pracy porównano wpływ integracji dodatkowych cech kategorycznych na skuteczność predykcyjną modeli. Całość uzupełniono ewaluacją subiektywną, mającą na celu weryfikację, w jakim stopniu wzbogacenie danych o kontekst pochodzenia artysty przekłada się na jakość i różnorodność rekomendacji postrzeganych przez użytkowników końcowych.

Proszę odnosić się do tej pracy używając następującego formatu referencji:

Maciej Andrzejewski, Wojciech Kaczmarek, Julia Nowakowska, Sławomir Stefański, and Sebastian Wróbel. Systemy rekomendacji personalizowanych na niewielkich zbiorach danych: analiza na przykładzie książek i muzyki. Politechnika Poznańska 2026 Raport z zajęć projektowych: Eksploracja Danych, Raport Końcowy.

Dostępność kodu:

Repozytorium kodu dostępne jest pod adresem:
<https://github.com/Narrovv/Projekt-Eksploracja-Danych>.

1 Wprowadzenie

Większość algorytmów rekomendacyjnych wymaga ogromnych zbiorów danych, co sprawia, że są one nieużyteczne dla indywidualnego użytkownika chcącego zarządzać własną, niszową kolekcją. Jest to istotne zagadnienie, ponieważ rozwój spersonalizowanych, lokalnych systemów

rekomendacyjnych pozwala na odzyskanie kontroli nad własnymi preferencjami bez polegania na zamkniętych algorytmach. Dodatkową motywacją jest możliwość wykorzystania modeli wytrenowanych na potężnych zbiorach danych do ekstrakcji istotnych cech, które następnie można zastosować w mniejszej, spersonalizowanej skali. Skuteczne przeniesienie metodologii z książek na inne dziedziny, takie jak muzyka, udowodni, że dobrze zaprojektowana inżynieria cech jest ważniejsza niż sama wielkość zbioru danych.

2 Powiązane Prace

Projektowanie personalnych systemów rekomendacyjnych na małych zbiorach danych (ang. *small datasets*) stanowi odrębne wyzwanie w dziedzinie eksploracji danych. W przeciwieństwie do systemów komercyjnych, które opierają się na analizie zachowań milionów ludzi i szukaniu wzorców oraz podobieństw między nimi, systemy spersonalizowane muszą wyekstrahować sygnał preferencji wyłącznie z ograniczonej historii jednej, konkretnej osoby.

2.1 Inspiracja metodologiczna: Model Habermaha

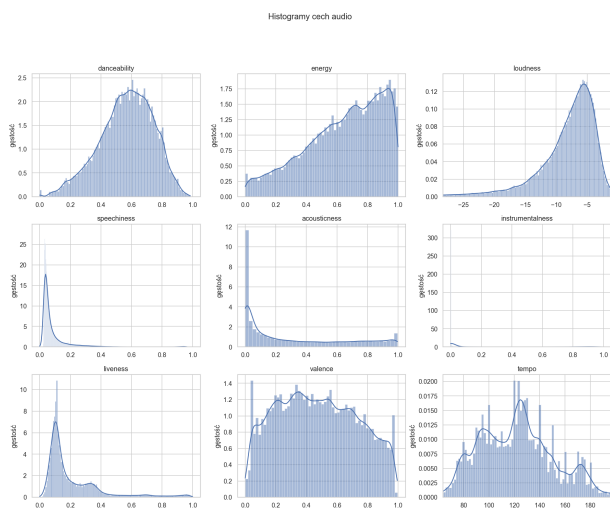
Bezpośrednią inspiracją dla naszego projektu był artykuł Davida Habermaha [1]. Autor opisał w nim, jak zbudował spersonalizowany model do przewidywania, czy polubi daną książkę, bazując na swojej 10-letniej historii z portalu Goodreads. Ponieważ dysponował niewielkim zbiorem danych, zamiast prognozować dokładne oceny w skali 1–5, uprościł problem do klasyfikacji binarnej. Podzielił książki na dwie grupy: pozycje „polubione” (oceny 4–5) oraz „niepolubione” (oceny 1–3). W naszym projekcie zaadaptowaliśmy tę samą strategię, przenosząc ją do domeny muzycznej i tworząc podobnie zbalansowany zbiór uczący. W swojej pracy Habermah zwrócił uwagę na bardzo ważny problem techniczny: próba łączenia danych z zewnętrznym OpenLibrary API za pomocą samych numerów ISBN dała tylko 15–20% udanych dopasowań. Wynikało to z tego, że numery ISBN dotyczą konkretnych, fizycznych wydań książki (różne okładki czy wydawnictwa), a nie samej powieści. Dopiero przejście na elastyczne wyszukiwanie po tytule i autorze podniosło skuteczność dopasowania do 95%. W domenie muzycznej zastosowaliśmy podobną logikę podczas łączenia danych o utworach ze Spotify [3] z bazą MusicBrainz [5]. Ponieważ bazy te operują na zupełnie innych identyfikatorach, nasz potok danych musiał opierać się na dopasowywaniu pól tekstowych (nazw wykonawców), co pozwoliło nam skutecznie powiązać profile użytkowników z krajem pochodzenia artystów.

2.2 Filtrowanie i modelowanie w domenie muzycznej

W domenie muzycznej standardowym podejściem jest filtrowanie oparte na zawartości (ang. *content-based filtering*), polegające na analizowaniu parametrów technicznych utworów. Wykorzystując bibliotekę *spotipy* [4], można bezpośrednio ze Spotify API [3] pobrać gotowe metadane charakteryzujące brzmienie, takie jak tempo (*tempo*), głośność (*loudness*) czy taneczność (*danceability*). Głównym wyzwaniem przy tej metodzie jest ryzyko zbytniego zawężenia rekomendacji do utworów o bardzo zbliżonej charakterystyce akustycznej. Rozwiązaniem tego problemu może być uzupełnienie cech audio o metadane kontekstowe opisujące wykonawców, pochodzące z otwartych baz wiedzy, takich jak MusicBrainz [5]. W warunkach małej liczby próbek stosowanie głębokich sieci neuronowych jest nieuzasadnione ze względu na ryzyko przeuczenia (*overfitting*). W ślad za podejściem Haberalha [1] do analizy wybrano dwa klasyczne algorytmy uczenia nadzorowanego. Pierwszym z nich jest Regresja Logistyczna [9] – model liniowy szacujący prawdopodobieństwo polubienia utworu, którego zaletą jest wysoka interpretowalność wag cech. Drugim rozwiązaniem jest Las Losowy (Random Forest) [8], pozwalający na wychwycenie bardziej skomplikowanych, nieliniowych zależności między zmiennymi. Do ewaluacji statystycznej obu podejść, wzorem literatury referencyjnej [1], wybrano metrykę ROC AUC [6]. Jest ona kluczowa w zadaniach rekomendacyjnych, ponieważ ocenia zdolność modelu do poprawnego sortowania (rankingu) obiektów, pozostając niezależną od przyjętego progu klasyfikacji.

3 Opis zbioru danych

W analizie wykorzystano globalny zbiór *Spotify Tracks Dataset* pobrany z platformy Kaggle [2]. Zbiór zawiera 114 000 rekordów opisujących utwory muzyczne oraz 21 kolumn, z czego 15 ma charakter numeryczny, a 6 jest kategorię lub tekstowych. Dane są uporządkowane według 114 gatunków muzycznych (*track_genre*), przy czym każdy gatunek reprezentowany jest przez dokładnie 1000 utworów, co oznacza, że zbiór jest zbalansowany klasowo. Po wczyceniu danych nie wykryto duplikatów. Braki danych wystąpiły jedynie w trzech kolumnach tekstowych: *artists*, *album_name* oraz *track_name*, i dotyczyły po jednym rekordzie w każdej z nich.

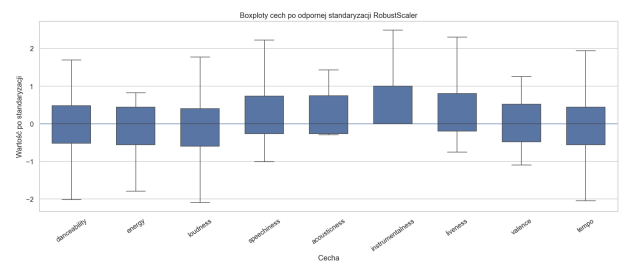


Rysunek 1: Histogramy cech audio w zbiorze Spotify Tracks Dataset.

W dalszej analizie skupiono się na dziewięciu cechach akustycznych wykorzystywanych jako wejście do modeli: *danceability*, *energy*, *loudness*, *speechiness*, *acousticness*, *instrumentalness*, *liveness*, *valence* oraz *tempo*.

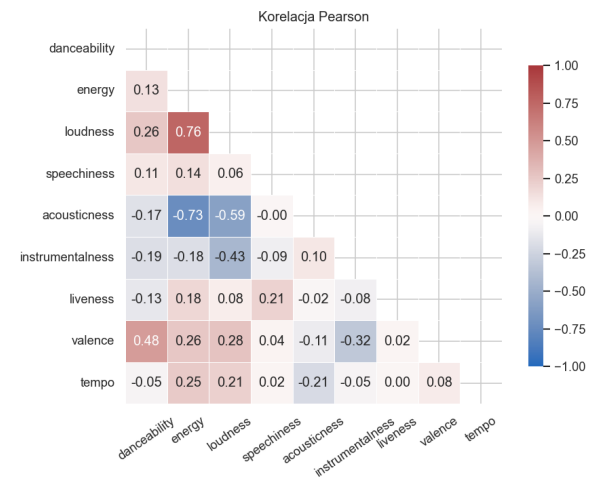
Rozkłady cech pokazano na Rysunku 1. Warto zauważyć, że atrybut tempo charakteryzuje się symetrycznym rozkładem zbliżonym do normalnego. Z kolei cechy takie jak *speechiness*, *acousticness*, *instrumentalness* oraz *liveness* wykazują wyraźną skośność prawostronną z długimi ogonami, co oznacza, że większość utworów przyjmuje w nich niskie wartości.

Analiza rozrzutu wartości i obecności obserwacji odstających dla cech audio została przedstawiona na Rysunku 2. Z powodu widocznych asymetrii oraz licznych punktów odstających w rozkładach, w dalszym przetwarzaniu danych zdecydowano się na zastosowanie odpornej standaryzacji *RobustScaler*, a nie zwykłego skalowania standardowego.

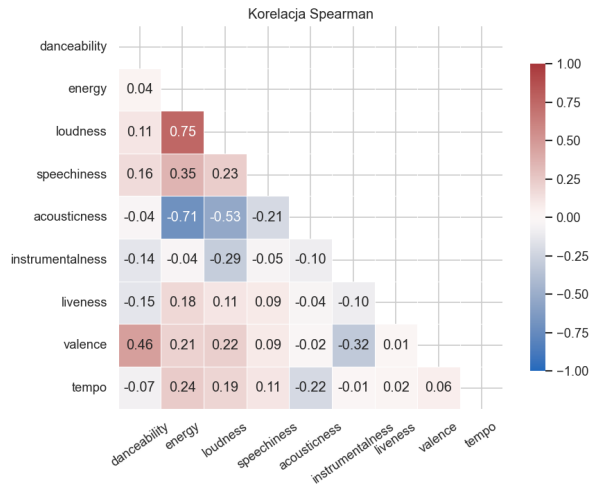


Rysunek 2: Wykresy pudełkowe cech audio po odpornej standaryzacji *RobustScaler*.

Analiza korelacji została przeprowadzona z wykorzystaniem współczynników Pearsona oraz Spearmana. Najsilniejszą dodatnią zależność zaobserwowano pomiędzy *energy* i *loudness*, natomiast najsilniejszą ujemną pomiędzy *energy* i *acousticness*. Pozostałe zależności są słabsze, co sugeruje, że cechy akustyczne niosą częściowo komplementarną informację. Macierze korelacji przedstawiono na Rysunkach 3 i 4.

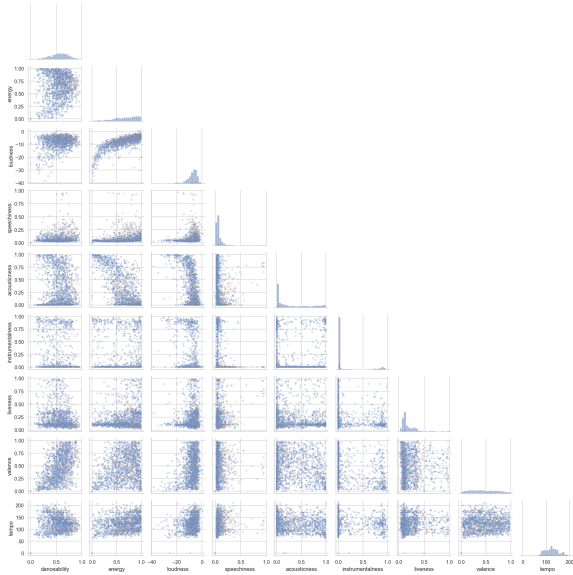


Rysunek 3: Macierz korelacji Pearsona dla wybranych cech audio.



Rysunek 4: Macierz korelacji Spearmana dla wybranych cech audio.

Dodatkowy wykres typu *pairplot* przedstawiono na Rysunku 5. Pokazuje on jednocześnie rozkłady wszystkich dziewięciu cech akustycznych oraz ich wzajemne zależności parami, z podziałem na zmienną binarną *explicit*, oznaczającą obecność jawnych treści w utworze. Dzięki temu można sprawdzić, czy utwory oznaczone jako *explicit* różnią się pod względem cech akustycznych od pozostałych nagrań. *Pairplot* pozwala lepiej ocenić kształt rozkładów oraz potwierdzić, że między częścią cech występują wyraźne zależności liniowe.



Rysunek 5: Wykres typu pairplot dla wszystkich cech audio z podziałem na explicit.

4 Opis Eksperymentów

W niniejszej sekcji przedstawiono architekturę stworzonego systemu rekomendacyjnego, szczegółowy przebieg przeprowadzonych iteracji

badawczych, dobrane hiperparametry oraz metody ewaluacji skuteczności modeli na podstawie metryk analitycznych i testów z udziałem użytkowników.

4.1 Opis eksperymentów

System bazuje na architekturze potokowej (ETL + Machine Learning). Dane wejściowe są ekstrahowane z prywatnych profili Spotify użytkowników (poprzez Spotify API i bibliotekę *spotipy*), a następnie łączone z globalną bazą utworów (Kaggle Dataset – 114 tys. utworów). Jako tło (przykłady negatywne) system losuje utwory nieznajdujące się na playlistach użytkownika, tworząc zbalansowany zbiór uczący w proporcji 50/50. W ramach projektu przeprowadzono dwa główne eksperymenty (iteracje), różniące się podejściem do inżynierii cech oraz obróbki danych:

- Eksperyment 1 (Model Bazowy): System oparty wyłącznie na cechach akustycznych (m.in. *danceability*, *energy*, *tempo*, *valence*). Zastosowano standaryzację danych (*StandardScaler*). Wyniki ukazały tendencję modelu do zamykania użytkownika w tzw. bańce filtrującej (rekomendowanie identycznie brzmiących utworów bez kontekstu kulturowego).
- Eksperyment 2 (Data Enrichment): Wzbogacenie zbioru danych o kontekst geograficzny. Wdrożono skrypt przetwarzający surowe zrzuty bazy MusicBrainz (pliki JSONL), łącząc artystów z krajami ich pochodzenia. Zmienną kategoryczną kraju zakodowano metodą *One-Hot Encoding*. Skaler zmieniono na *RobustScaler* w celu lepszej odporności na wartości odstające w cechach akustycznych. Dodano mechanizm wyrównywania wymiarów (*.reindex()*) dla nowych danych w fazie predykcji.

W obu eksperymentach trenowano modele *Random Forest* oraz *Logistic Regression*. Wymienione hiperparametry dobrane w celu ograniczenia zjawiska przeuczenia (*overfitting*).

Tabela 1: Konfiguracja i hiperparametry w eksperymentach

Komponent	Parametry (Eksperyment 1)	Parametry (Eksperyment 2)
Zbiór uczący	Akustyka dźwięku (11 cech)	Akustyka + Kraj (One-Hot Encoding)
Skalowanie danych	<i>StandardScaler()</i>	<i>RobustScaler()</i>
Random Forest	<i>n_estimators=100</i> , <i>max_depth=5</i> , <i>class_weight='balanced'</i>	<i>n_estimators=100</i> , <i>max_depth=5</i> , <i>class_weight='balanced'</i>
Regresja Logistyczna	<i>max_iter=1000</i> , <i>class_weight='balanced'</i>	<i>max_iter=1000</i> , <i>class_weight='balanced'</i>
Walidacja modelu	5-krotna walidacja krzyżowa (CV)	5-krotna walidacja krzyżowa (CV)

4.2 Ewaluacja

Do oceny jakości modeli wykorzystano dwa podejścia: ewaluację analityczną (statystyczną) oraz ewaluację subiektywną z udziałem grupy badawczej.

- (1) ROC AUC (Area Under the Receiver Operating Characteristic Curve): Główna metryka techniczna. Wskazuje, jak dobrze model radzi sobie z rozróżnianiem utworów polubionych od tła losowego. Wynik 1.0 oznacza idealną separację, a 0.5 działanie losowe. Wybrano tę metrykę, ponieważ jest ona niezależna od przyjętego progu prawdopodobieństwa.
- (2) Dokładność (Accuracy): Odsetek poprawnie sklasyfikowanych próbek w zbiorze testowym. Mając sztucznie zbalansowany zbiór

(50/50), metryka ta jest miarodajna i wspiera interpretację ROC AUC.

- (3) Trafność Rekomendacji (Hit Rate / User Acceptance):
Najważniejsza, biznesowa metryka wdrożona w tym projekcie. Każdy wygenerowany model zwraca zestawienie Top 10 rekomendacji dla danego użytkownika. Ocenie podlega to, ile z dziesięciu zaproponowanych, nieznanymi wcześniej utworów, faktycznie przypadło do gustu użytkownikowi.

4.3 Rezultaty

Wyniki techniczne modeli wskazały na wysoką skuteczność analityczną już w pierwszej iteracji. Wzbogacenie danych w drugiej iteracji nie zmieniło drastycznie metryk statystycznych, ale znacząco wpłynęło na jakość rekomendacji odczuwaną przez użytkowników.

Tabela 2: Analityczne wyniki modeli klasyfikujących

Iteracja	Algorytm	Dokładność (Acc)	ROC AUC (Test)	ROC AUC (CV)
Iteracja 1	Random Forest	0.80	0.90	-
Iteracja 1	Regresja Logistyczna	-	-	0.82
Iteracja 2	Random Forest	0.86	0.92	-
Iteracja 2	Regresja Logistyczna	-	-	0.82

W celu zmierzenia realnej wartości systemu, każdy członek zespołu wygenerował rekomendacje na podstawie swoich prywatnych playlist. Poniższa tabela przedstawia odsetek utworów z Top 10, które po odsłuchaniu zostały ocenione jako "trafione".

Tabela 3: Ewaluacja subiektywna (Hit Rate w Top 10 rekomendacji)

Użytkownik	Iteracja 1 (Cechy akustyczne)	Iteracja 2 (Akustyka + Kraj)
Maciej	60% (6/10)	80% (8/10)
Wojciech	60% (6/10)	80% (8/10)
Julia	50% (5/10)	80% (8/10)
Sławek	20% (2/10)	60% (6/10)
Sebastian	50% (5/10)	60% (6/10)
Średnia	48% (4.8/10)	72% (7.2/10)

Mimo że metryki analityczne wzrosły nieznacznie, każdy z ankietowanych użytkowników odnotował wyraźny wzrost wskaźnika trafności (Hit Rate).

5 Prace rozwojowe

W przyszłości system może zostać rozwinięty o następujące elementy:

- Analiza Tekstu (NLP): Zastosowanie metod przetwarzania języka naturalnego na warstwie tekstowej utworów w celu wychycenia tematów i ładunku emocjonalnego.
- Collaborative Filtering: Obecny model opiera się w całości na filtrowaniu po zawartości (Content-Based). Dodanie logiki CF (bazującej na podobieństwie między różnymi użytkownikami systemu) pozwoliłoby na wdrożenie podejścia hybrydowego.
- Wdrożenie produkcyjne: Refaktoryzacja obecnych skryptów Python do postaci API i osadzenie modelu w prostej aplikacji webowej z frontendem, pozwalającej na logowanie się Spotify OAuth bezpośrednio z poziomu przeglądarki.

6 Konkluzje

Projekt wykazał, że zbudowanie skutecznego systemu rekomendacyjnego nie opiera się wyłącznie na skomplikowaniu algorytmów uczenia maszynowego, ale w dużej mierze na jakości i wymiarowości danych. Podstawowa iteracja udowodniła skuteczność klasyfikatora Random Forest w modelowaniu cech fizycznych dźwięku, jednak wpadła w pułapkę "bańki akustycznej". Rozwiązaniem tego problemu okazało się wzbogacenie danych (Data Enrichment). Skonstruowanie lokalnego potoku ETL przetwarzającego surowe zrzuty bazy MusicBrainz umożliwiło nadanie utworom niezbędnego kontekstu kulturowego. Mimo, że dodanie kilkudziesięciu nowych cech (krajów) nie zwiększyło drastycznie wskaźników statystycznych modelu, to zdecydowanie poprawiło odczuwalną, subiektywną jakość rekomendacji w testach grupowych. Wskazuje to na to, że w dziedzinach sztuk takich jak muzyka, modele czysto matematyczne powinny być wspomagane danymi semantycznymi i pochodzeniowymi.

References

- [1] Haberlah, D.: Building my own Book Recommendation Engine. Medium (2025), <https://medium.com/@haberlah/building-my-personal-book-recommendation-engine-0d103180d56b>
- [2] Pandya, M.: Spotify Tracks Dataset. Kaggle, <https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset>
- [3] Spotify: Web API Documentation, <https://developer.spotify.com/documentation/web-api/>
- [4] Spotipy: Python library for the Spotify Web API, <https://spotipy.readthedocs.io/>
- [5] MusicBrainz: Database Documentation & Dumps, https://musicbrainz.org/doc/MusicBrainz_Database
- [6] Scikit-learn: ROC AUC score documentation, https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html
- [7] Scikit-learn: NDCG score documentation, https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ndcg_score.html
- [8] GeeksforGeeks: Random Forest Regression in Python, <https://www.geeksforgeeks.org/machine-learning/random-forest-regression-in-python/>
- [9] GeeksforGeeks: Logistic Regression in Python, <https://www.geeksforgeeks.org/machine-learning/ml-logistic-regression-using-python/>